

Énoncé — TP 3 : L'entrepôt résilient (HDFS & Spark)

Une plateforme e-commerce archivait jusqu'ici ses journaux de commandes sur le disque d'une seule machine. Un incident récent (disque plein, un jour de forte affluence) a convaincu la direction technique de revoir l'architecture : les données brutes doivent désormais être stockées sur un cluster distribué et tolérant aux pannes, et c'est ce cluster que Spark doit interroger directement pour produire les rapports — plus aucun fichier ne doit transiter par le disque local d'une seule machine.

Données fournies

Le script suivant génère **trois fichiers** `commandes_2026-06-12.csv`, `commandes_2026-06-13.csv`, `commandes_2026-06-14.csv` (1000 lignes chacun, colonnes : `id_commande`, `date`, `client_id`, `produit`, `categorie`, `quantite`, `prix_unitaire`, `entrepot`), simulant les journaux quotidiens d'une plateforme avec plusieurs entrepôts.

```
import csv, random
random.seed(7)

produits = {
    "Casque audio": ("Electronique", 45, 120),
    "Livre": ("Culture", 8, 25),
    "Chaise bureau": ("Mobilier", 60, 250),
    "Plante verte": ("Maison", 12, 40),
    "Console jeux": ("Electronique", 200, 450),
}

entrepots = ["Entrepot Nord", "Entrepot Sud", "Entrepot Est"]
dates = ["2026-06-12", "2026-06-13", "2026-06-14"]

for date in dates:
    rows = []
    for i in range(1, 1001):
```

```

produit = random.choice(list(produits.keys()))
categorie, lo, hi = produits[produit]
quantite = random.randint(1, 5)
prix = round(random.uniform(lo, hi), 2)
entrepot = random.choices(entrepots, weights=[0.5, 0.3, 0.2])[0]
rows.append([f"C{date.replace('-', '')}{i:04d}", date,
f"CL{random.randint(1,400):04d}",
               produit, categorie, quantite, prix, entr
epot])
with open(f"commandes_{date}.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["id_commande", "date", "client_id", "produi
t", "categorie", "quantite", "prix_unitaire", "entrepot"])
    w.writerows(rows)

```

Ce script tourne sur la machine hôte (pas dans un conteneur) : les fichiers générés doivent ensuite être **chargés dans HDFS**, pas simplement partagés via un volume comme dans le TP précédent.

Ce qu'on attend de vous

- Monter un cluster HDFS avec Docker Compose : 1 namenode et **3 datanodes minimum**, facteur de répllication d'au moins 3.
- Charger les trois fichiers CSV dans HDFS et vérifier qu'ils sont bien découpés en blocs répartis sur plusieurs datanodes.
- Monter un cluster Spark standalone, connecté au même réseau que le cluster HDFS, capable de lire et écrire directement en `hdfs://...` (sans passer par un volume partagé ni par un chemin local).
- Lire l'ensemble des trois fichiers en un seul DataFrame Spark, avec le bon typage des colonnes.
- Calculer, par entrepôt et par jour : le chiffre d'affaires total, le nombre de commandes, et le panier moyen.
- Écrire le résultat agrégé dans HDFS, au format **Parquet**.

- Pendant qu'un job Spark tourne sur les données, **arrêter un des datanodes** et observer/documenter ce qu'il se passe : le job échoue-t-il ? réussit-il ? pourquoi ?
- Fournir une preuve de la réplication des blocs, avant et après l'arrêt du datanode.

Rendu attendu

Un git avec un readme qui documente chaque étape.